

Weight-Space Divergence and Loss Landscape Connectivity in Domain-Specialized Fine-Tuning

Draft v1 — July 5, 2026

Abstract

We measure the relationship between domain-specific fine-tuning and weight-space divergence in Pythia-1.4B, and test whether Linear Mode Connectivity (LMC) holds between models fine-tuned on different domains. Full-parameter fine-tuning on code and medical reasoning tasks for 1–5 epochs produces only 0.26%–1.95% mean weight divergence from the pretrained checkpoint. LMC barrier scans across three training intensities (1, 3, 5 epochs) reveal barriers ≤ 0.08 — well within the linearly-connected regime. We observe an asymmetric U-shaped cross-domain transfer: code model weights improve undertrained medical model performance (up to 8.3% loss reduction) but not vice versa. This benefit vanishes as the medical model is trained longer, with the 5-epoch model achieving optimal performance without any code knowledge. We hypothesize that the small weight divergence is a function of the learning rate (1×10^{-4}) rather than an inherent property of the model, and propose experiments to test this.

1 Introduction

Fine-tuning pretrained language models on domain-specific data is standard practice. But how far do these models actually move in weight space? And when two models are fine-tuned on different domains, does the linear path between their weights cross a loss barrier?

Linear Mode Connectivity [1] established that neural networks fine-tuned from the same initialization often lie in the same linearly-connected loss basin. However, the relationship between the *amount* of fine-tuning (epochs, learning rate, data volume) and the *magnitude* of weight-space divergence remains underexplored.

We conduct a systematic measurement on Pythia-1.4B, fine-tuning two variants on code reasoning (VersaPRM-code) and medical reasoning (VersaPRM-medical). We measure:

1. Per-block weight divergence from the pretrained checkpoint
2. LMC barrier via 11-point linear interpolation between domain-specialized models
3. Cross-domain transfer asymmetry at three medical training intensities

2 Method

2.1 Training

Table 1: Training configuration

Parameter	Value
Base model	EleutherAI/pythia-1.4b (1.31B params)
Training	Full-parameter fine-tuning
Data	VersaPRM: code (55K train) + medical (~55K train)
Precision	bfloat16 autocast, fp32 classification head
Batch size	128
Sequence length	256 tokens
Learning rate	Cosine $1 \times 10^{-4} \rightarrow 3 \times 10^{-6}$
Epochs	1 (code), 1/3/5 (medical)
Optimizer	AdamW ($\beta = 0.9, 0.999$, wd= 0.1)
Hardware	NVIDIA DGX Spark GB10 (121GB unified, ARM64 CUDA 13.0)

2.2 LMC Barrier Scan

For each model pair $(\theta_{\text{code}}, \theta_{\text{med_eN}})$, we construct 11 interpolated models:

$$\theta(\alpha) = (1 - \alpha) \cdot \theta_{\text{code}} + \alpha \cdot \theta_{\text{med}}, \quad \alpha \in \{0.0, 0.1, \dots, 1.0\} \quad (1)$$

We evaluate the binary cross-entropy loss of each interpolated model on both code and medical test data (2,000 samples each). The LMC barrier is defined as:

$$\text{barrier} = \max_{\alpha} \mathcal{L}(\theta(\alpha)) - \mathcal{L}(\theta(0)) \quad (2)$$

2.3 Weight Divergence

Per-block relative L2 divergence from base model:

$$\Delta_{\text{block } j} = \frac{\|W_{\text{trained, block } j} - W_{\text{base, block } j}\|}{\|W_{\text{base, block } j}\|} \quad (3)$$

3 Results

3.1 Weight Divergence Is Small

Table 2: Weight divergence and self-domain accuracy

Model	Epochs	Mean $\ \Delta W\ /\ W\ $	Max $\ \Delta W\ /\ W\ $	Self-domain acc.
code	1	0.26%	6.2%	0.751
medical	1	1.91%	13.5%	0.882
medical	3	1.95%	14.3%	0.885
medical	5	1.95%	14.5%	0.886

Key observations:

- Code fine-tuning produces $7.4\times$ less weight change than medical (0.26% vs 1.91%)
- Medical training saturates after epoch 1: epochs 2–5 add only 0.04% total divergence
- Despite different magnitudes, per-block importance patterns are highly correlated (cosine > 0.94)

3.2 LMC Holds Across All Model Pairs

Table 3: LMC barrier scan results

Med epoch	$\mathcal{L}_{\text{code}}(0)$	Code barrier	$\mathcal{L}_{\text{med}}(1)$	Med U-min	Cross benefit
1	0.6155	0.071	0.623	0.495 ($\alpha=0.2$)	8.3%
3	0.6155	0.076	0.507	0.503 ($\alpha=0.2$)	6.8%
5	0.6155	0.076	0.485	0.485 ($\alpha=1.0$)	0%

All barriers < 0.08 across both domains. LMC is preserved for all training intensities.

3.3 Asymmetric U-Shaped Cross-Domain Transfer

Finding 1: Code knowledge transfers to medical, not vice versa.

The code model’s loss on medical data (0.540) is *lower* than base Pythia’s (0.559). Code fine-tuning produces a positive externality for medical reasoning. Medical fine-tuning produces no reciprocal benefit for code.

Finding 2: The benefit decays with medical training intensity.

At 1 epoch, mixing 20% code weights reduces medical loss by 8.3% (U-shaped optimum at $\alpha = 0.2$). At 3 epochs, the benefit drops to 6.8%. At 5 epochs, the medical model’s own weights ($\alpha = 1.0$) are optimal — the U-shape vanishes.

Finding 3: Code is always harmed by medical mixing.

Code domain loss increases monotonically with α for all medical model variants. Medical knowledge does not generalize to code reasoning.

3.4 Training Speed

On DGX Spark GB10, full fine-tuning reaches 5.5 seconds per training step (batch=128, seq=256). One epoch (433 steps) completes in ~ 42 minutes.

4 Discussion

4.1 Why Is Pythia So Stubborn?

We hypothesize that the small weight divergence is primarily a function of the **learning rate** (1×10^{-4}), not an inherent property of the model architecture or pretraining data. At $\text{lr} = 10^{-4}$ with cosine decay to 3×10^{-6} , the average per-step update magnitude is small. With only 433 steps per epoch, the cumulative weight change is bounded.

Supporting evidence:

- Medical training (more gradient signal from clearer labels) produces $7\times$ more divergence than code training

- Within medical training, epochs 2–5 add negligible divergence — the model reaches a flat region of the loss basin where gradient magnitude approaches zero
- The barrier magnitude (0.07–0.08) is consistent across model pairs despite different training intensities

Prediction: Training at $\text{lr} = 5 \times 10^{-4}$ or 10^{-3} , or with more data per epoch, will produce significantly larger weight divergence and potentially break LMC.

4.2 Comparison to Prior Work

Frankle et al. (2020) demonstrated LMC for models fine-tuned on the *same* task with different random seeds. Our results extend this to *different domains* — even when models are optimized for qualitatively different capabilities, they remain in the same loss basin.

The U-shaped cross-domain transfer curve is consistent with the “model merging” literature [2, 3, 4], where small amounts of weight interpolation can improve performance through implicit regularization. Our contribution is showing this effect is (a) asymmetric between domains and (b) training-intensity dependent.

4.3 Limitations

1. Single model architecture (Pythia-1.4B). Results may not generalize to other scales or architectures.
2. Single training run per condition. No statistical significance estimates.
3. Only two domains (code, medical). Additional domains needed to characterize the asymmetry.
4. Limited training intensity range. The LR hypothesis (§4.1) is untested.

5 Next Experiments

Priority 1: Test the LR Hypothesis

Train code and medical at $\text{lr} \in \{5 \times 10^{-4}, 10^{-3}, 2 \times 10^{-3}\}$ for 1 epoch. Measure weight divergence and re-run LMC scan. **Prediction:** $\text{lr} \geq 5 \times 10^{-4}$ will produce $> 5\%$ weight divergence and may break LMC.

Priority 2: Statistical Significance

Repeat the $\text{lr} = 10^{-4}$ condition with 3 random seeds. Report mean $\pm$ std for all metrics.

Priority 3: Scale Variation

Repeat on Pythia-160M and Pythia-2.8B to test whether stubbornness scales with model size.

References

- [1] J. Frankle, G. K. Dziugaite, D. M. Roy, and M. Carbin. Linear Mode Connectivity and the Lottery Ticket Hypothesis. In *ICML*, 2020.
- [2] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt. Model soups: averaging weights of multiple fine-tuned models. In *ICML*, 2022.

- [3] M. Wortsman, G. Ilharco, J. W. Kim, M. Li, S. Kornblith, R. Roelofs, R. Gontijo-Lopes, H. Hajishirzi, A. Farhadi, H. Namkoong, and L. Schmidt. Robust fine-tuning of zero-shot models. In *CVPR*, 2022.
- [4] P. Yadav, D. Tam, L. Choshen, C. Raffel, and M. Bansal. TIES-Merging: Resolving Interference When Merging Models. In *NeurIPS*, 2023.
- [5] S. Biderman, H. Schoelkopf, Q. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, A. Skowron, S. Sutawika, and O. van der Wal. Pythia: A Suite for Analyzing Large Language Models. In *NeurIPS*, 2023.